

Scalar UDF Parallelism Barrier

#sql-server #performance #parallelism #refactoring

The Problem

In SQL Server (pre-2019, or 2019+ without scalar UDF inlining enabled), a scalar user-defined function (UDF) in a query forces the entire query to run in a serial plan. The optimizer will not consider parallel execution for any part of the query if a scalar UDF is present anywhere in the SELECT list, WHERE clause, or JOIN conditions.

This is one of the most impactful and least-understood performance killers in SQL Server. A query that should take 200ms across 8 cores instead takes 1,600ms on a single core, an 8x penalty, because someone called a scalar function.

Why Serial?

Scalar UDFs are essentially black boxes to the optimizer. Each invocation:

1. Creates a new execution context (T-SQL interpreter frame).
2. May perform its own data access (queries inside the function).
3. Cannot be safely distributed across parallel threads without risking incorrect results.

Rather than deal with the complexity of coordinating UDF execution across threads, the optimizer simply disables parallelism for the entire statement.

How to Detect It

In the execution plan:

- Look for NonParallelPlanReason="CouldNotGenerateValidParallelPlan" or similar in the plan XML.
- The plan will show no Parallelism (Gather Streams) operators.
- SET STATISTICS TIME ON will show CPU time at or above elapsed time, which signals serial execution. In a parallel plan, elapsed is below CPU because multiple cores share the work.

In the code:

- Look for `dbo.functionName(...)` calls in SELECT lists.
- Any function that returns a scalar value (not a table) is suspect.

```
-- This entire query runs serial because of the scalar UDF.  
Select  
    G.GroupNum,  
    G.GroupingCriteriaVal,  
    dbo.f0rdGroupingCritValForDisplay(G.GroupNum) As FormattedDisplay -- SERIAL KILLER  
From OeGroup G
```

The Fix: Inline the Logic

Replace the scalar UDF call with equivalent set-based logic directly in the query, typically as a CTE or a derived table with a LEFT JOIN.

Before (Scalar UDF)

```
-- The function: cursor-style string concatenation
Create Function dbo.f0rdGroupingCritValForDisplay(@GroupNum varchar(10))
Returns varchar(200)
As Begin
    Declare @Result varchar(200), @Suffix varchar(100)
    Select @Result = GroupingCriteriaVal From OeGroup Where GroupNum = @GroupNum

    Select @Suffix = Coalesce(@Suffix + '; ', '') + SC.Descrip
    From OeGroupSplitCodeAssoc GSA
    Join OeGroupSplitCode SC On SC.Code = GSA.SplitCode
    Where GSA.GroupNum = @GroupNum

    If @Suffix Is Not Null
        Set @Result = @Result + ' (' + @Suffix + ')'

    Return @Result
End
```

After (Inlined with STRING_AGG)

```
;With GroupingCriteria As (
    Select
        DG.GroupNum,
        G.GroupingCriteriaVal
        + IsNull(' (' + S.SplitSuffix + ')', '')
        As FmtdGroupingCriteriaVal
    From DistinctGroups DG
    Join OeGroup G With (NoLock) On G.GroupNum = DG.GroupNum
    Left Join (
        Select
            GSA.GroupNum,
            String_Agg(SC.Descrip, '; ') As SplitSuffix
        From OeGroupSplitCodeAssoc GSA With (NoLock)
        Join OeGroupSplitCode SC With (NoLock) On SC.Code = GSA.SplitCode
        Group By GSA.GroupNum
    ) S On S.GroupNum = DG.GroupNum
)
```

Key techniques in the inlining:

1. STRING_AGG replaces the cursor-style @Suffix = Coalesce(@Suffix + '; ', '') + Descrip concatenation. STRING_AGG is set-based and available in SQL Server 2017 and later.
2. The subquery with GROUP BY handles the split code aggregation for all groups at once instead of per-row.
3. The CTE makes the result available for a simple LEFT JOIN into the main query.
4. Scope narrowing: the DistinctGroups CTE (computed earlier in the query) limits the function's work to only groups actually in the result set, rather than every group in the database.

STRING_AGG: The Scalar UDF Killer

STRING_AGG (SQL Server 2017+) is the single most common tool for inlining scalar UDFs, because an enormous number of scalar UDFs exist solely to do string concatenation:

```

-- Old pattern (pre-2017): FOR XML PATH
Select Stuff(
    (Select ';' + SC.Descrip
     From OeGroupSplitCode SC
     Where SC.GroupNum = @GroupNum
     For Xml Path(''), Type).value('.', 'varchar(max)')
, 1, 2, '')

-- New pattern (2017+): STRING_AGG
Select String_Agg(SC.Descrip, '; ')
From OeGroupSplitCode SC
Where SC.GroupNum = @GroupNum

```

Verifying Parallelism Is Restored

After inlining, check with SET STATISTICS TIME ON:

```

-- Serial (UDF present):
-- CPU time = 1829 ms, elapsed time = 2099 ms

-- Parallel (UDF removed):
-- CPU time = 1829 ms, elapsed time = 890 ms

```

When elapsed is below CPU, you know multiple cores are working. The CPU total goes up slightly (coordination overhead) but wall clock time drops dramatically.

SQL Server 2019+ Scalar UDF Inlining

SQL Server 2019 introduced automatic scalar UDF inlining under compatibility level 150. The optimizer can sometimes inline the UDF itself without code changes. There are caveats, though:

- Not all UDFs qualify (no loops, no table variables, no EXEC, etc.).
- The feature can be disabled at database or function level.
- You can't always upgrade compatibility level.
- Manual inlining gives you control over the exact plan shape.

Manual inlining remains the reliable approach.

Real-World Example

In `isp_RxfGetListOfManualFillGroups` (v48 to v49), `dbo.f0rdGroupingCritValForDisplay` was called per-row in the enrichment query. The function contained two separate queries (one to `OeGroup`, one joining `OeGroupSplitCodeAssoc` to `OeGroupSplitCode`) and cursor-style string concatenation. Inlining it as a CTE with `STRING_AGG` restored parallelism to the entire enrichment query. `SET STATISTICS TIME` confirmed elapsed time dropped below CPU time, which is proof that parallelism was restored.

Related Concepts

- **Correlated Subqueries to CTEs**: same per-row problem, same CTE solution pattern.
- **Parameter Sniffing**: another plan-quality issue, but about estimation rather than parallelism.
- **Query Store Triage**: high CPU time with CPU at or above elapsed is a signal to look for scalar UDFs.